

Theory of Computation

Regular Languages and Regular Expressions

Samit Biswas¹

¹Department of Computer Science and Technology,
Indian Institute of Engineering Science and Technology, Shibpur
Email: samit@cs.iiests.ac.in

Table of Contents

1 Regular Expressions

Regular Expressions

- The Language accepted by **Finite Automata** are easily described by simple expressions called **Regular Expressions**.

Regular Expressions

- The Language accepted by **Finite Automata** are easily described by simple expressions called **Regular Expressions**.
- **Regular Expression** is a way to represent a language.

Regular Expressions

- The Language accepted by **Finite Automata** are easily described by simple expressions called **Regular Expressions**.
- **Regular Expression** is a way to represent a language.
- **Regular expressions** describe **Regular language**.

Regular Expressions

- The Language accepted by **Finite Automata** are easily described by simple expressions called **Regular Expressions**.
- **Regular Expression** is a way to represent a language.
- **Regular expressions** describe **Regular language**.
- **Example:** $(a + bc)^*$
describes the language
 $\{a, bc\}^* = \{\lambda, a, bc, aa, abc, bca, \dots\}$

Regular Expression

- **Recursive Definition:**

- Primitive regular expressions: Φ, λ, α

- **Recursive Definition:**

- Primitive regular expressions: Φ, λ, α
- Given Regular Expressions: r_1 & r_2

Regular Expression

- **Recursive Definition:**

- Primitive regular expressions: Φ, λ, α
- Given Regular Expressions: r_1 & r_2

$$\left. \begin{array}{l} r_1 + r_2 \\ r_1 \cdot r_2 \\ r_1^* \\ (r_1) \end{array} \right\} \text{Are regular expressions}$$

Regular Expression

- **Recursive Definition:**

- Primitive regular expressions: Φ, λ, α
- Given Regular Expressions: r_1 & r_2

$$\left. \begin{array}{l} r_1 + r_2 \\ r_1 \cdot r_2 \\ r_1^* \\ (r_1) \end{array} \right\} \text{Are regular expressions}$$

- A regular expression: $(a + bc)^*(c + \Phi)$

Regular Expression

- **Recursive Definition:**

- Primitive regular expressions: Φ, λ, α
- Given Regular Expressions: r_1 & r_2

$$\left. \begin{array}{l} r_1 + r_2 \\ r_1 \cdot r_2 \\ r_1^* \\ (r_1) \end{array} \right\} \text{Are regular expressions}$$

- A regular expression: $(a + bc)^*(c + \Phi)$
- Not a regular expression: $(a+b+)$

Languages of Regular Expressions

- $L(R)$: language of regular expression, r

Example: $L((a + b + c)^*) = \{\lambda, a, bc, aa, abc, bca, \dots\}$

Languages of Regular Expressions

- $L(R)$: language of regular expression, r

Example: $L((a + b + c)^*) = \{\lambda, a, bc, aa, abc, bca, \dots\}$

- For primitive regular expressions:

- $L(\Phi) = \Phi$
- $L(\lambda) = \{\lambda\}$
- $L(a) = \{a\}$
- $L(r_1 + r_2) = L(r_1) \cup L(r_2)$
- $L(r_1.r_2) = L(r_1)L(r_2)$
- $L(r_1^*) = (L(r_1))^*$
- $(L(r_1)) = L(r_1)$

Languages of Regular Expressions

- **Example:**

Regular Expression: $(a + b).a^*$

$$\begin{aligned} L((a + b).a^*) &= L(a + b)L(a^*) = (L(a) \cup L(b))L(a^*) = \\ &(\{a\} \cup \{b\})\{(a)^*\} = \{a, b\}\{\lambda, a, aa, aaa, \dots\} = \\ &\{a, aa, aaa, \dots, b, bb, bbb, \dots\} \end{aligned}$$

Languages of Regular Expressions

- **Example:**

Regular Expression: $(a + b).a^*$

$$\begin{aligned} L((a + b).a^*) &= L(a + b)L(a^*) = (L(a) \cup L(b))L(a^*) = \\ &(\{a\} \cup \{b\})\{(a)\}^* = \{a, b\}\{\lambda, a, aa, aaa, \dots\} = \\ &\{a, aa, aaa, \dots, b, bb, bbb, \dots\} \end{aligned}$$

- **Regular Expression:** $r = (a + b)^*(a + bb)$

$$L(r) = \{a, bb, aa, abb, ba, bbb, \dots\}$$

Languages of Regular Expressions

- **Example:**

Regular Expression: $(a + b).a^*$

$$\begin{aligned} L((a + b).a^*) &= L(a + b)L(a^*) = (L(a) \cup L(b))L(a^*) = \\ &(\{a\} \cup \{b\})\{(a)\}^* = \{a, b\}\{\lambda, a, aa, aaa, \dots\} = \\ &\{a, aa, aaa, \dots, b, bb, bbb, \dots\} \end{aligned}$$

- **Regular Expression:** $r = (a + b)^*(a + bb)$

$$L(r) = \{a, bb, aa, abb, ba, bbb, \dots\}$$

- **Regular Expression:** $r = (aa)^*(bb)^*b$

$$L(r) = \{a^{2n}b^{2m}b : n, m \geq 0\}$$

Languages of Regular Expressions

- **Example:**

Regular Expression: $(a + b).a^*$

$$\begin{aligned} L((a + b).a^*) &= L(a + b)L(a^*) = (L(a) \cup L(b))L(a^*) = \\ &(\{a\} \cup \{b\})\{(a)^*\} = \{a, b\}\{\lambda, a, aa, aaa, \dots\} = \\ &\{a, aa, aaa, \dots, b, bb, bbb, \dots\} \end{aligned}$$

- **Regular Expression:** $r = (a + b)^*(a + bb)$

$$L(r) = \{a, bb, aa, abb, ba, bbb, \dots\}$$

- **Regular Expression:** $r = (aa)^*(bb)^*b$

$$L(r) = \{a^{2n}b^{2m}b : n, m \geq 0\}$$

- **Regular Expression:** $r = (0 + 1)^*00(0 + 1)^*$

$$L(r) = \{\text{all strings with at least two consecutive } 0\}$$

Languages of Regular Expressions

- **Example:**

Regular Expression: $(a + b).a^*$

$$\begin{aligned} L((a + b).a^*) &= L(a + b)L(a^*) = (L(a) \cup L(b))L(a^*) = \\ &= (\{a\} \cup \{b\})\{(a)^*\} = \{a, b\}\{\lambda, a, aa, aaa, \dots\} = \\ &= \{a, aa, aaa, \dots, b, bb, bbb, \dots\} \end{aligned}$$

- **Regular Expression:** $r = (a + b)^*(a + bb)$

$$L(r) = \{a, bb, aa, abb, ba, bbb, \dots\}$$

- **Regular Expression:** $r = (aa)^*(bb)^*b$

$$L(r) = \{a^{2n}b^{2m}b : n, m \geq 0\}$$

- **Regular Expression:** $r = (0 + 1)^*00(0 + 1)^*$

$$L(r) = \{\text{all strings with at least two consecutive } 0\}$$

- **Regular Expression:** $r = (1 + 01)^*(0 + \lambda)$

$$L(r) = \{\text{all strings without two consecutive } 0\}$$

Equivalent Regular Expressions

- Regular expressions r_1 and r_2 are equivalent if $L(r_1) = L(r_2)$
- **Example:** $L = \{ \text{all strings without two consecutive 0} \}$
 $r_1 = (0 + 01)^*(0 + \lambda)$
 $r_2 = (1^*011^*)^*(0 + \lambda) + 1^*(0 + \lambda)$
 $L(r_1) = L(r_2) = L \implies r_1 \text{ and } r_2 \text{ are equivalent regular expression.}$

Regular Expressions and Regular Languages

Languages Generated by
by
Regular Expressions } = { **Regular Languages** }

Regular Expressions and Regular Languages

Languages Generated by
by
Regular Expressions $\Bigg\} = \{\textbf{Regular Languages}\}$

- **Theorem 1:** For any regular expression r the language $L(r)$ is regular.
- **Theorem 2:** For any regular language, $L(r)$, there is a regular expression r with $L(r) = L$.

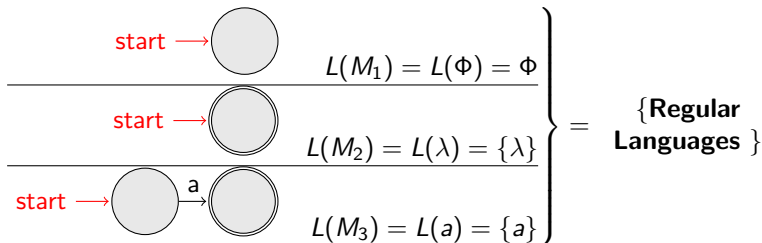
Regular Expressions and Regular Languages

- **Proof of Theorem 1:** For any regular expression r the language $L(r)$ is regular.

Proof by induction on the size of r .

- **Induction Basis:**

Primitive Regular Expressions: Φ, λ, a



Regular Expressions and Regular Languages

- **Inductive Hypothesis:** Assume for regular expressions r_1 and r_2 that $L(r_1)$ and $L(r_2)$ are regular languages.
 - **Inductive Step:** We will prove:

$$\left. \begin{array}{l} L(r_1 + r_2) \\ L(r_1 \cdot r_2) \\ L(r_1^*) \\ L((r_1)) \end{array} \right\} = \{\text{Regular Languages}\}$$

Regular Expressions and Regular Languages

- **Inductive Hypothesis:** Assume for regular expressions r_1 and r_2 that $L(r_1)$ and $L(r_2)$ are regular languages.
 - **Inductive Step:** We will prove:

$$\left. \begin{array}{l} L(r_1 + r_2) \\ L(r_1.r_2) \\ L(r_1^*) \\ L((r_1)) \end{array} \right\} = \{\text{Regular Languages}\}$$

By definition of regular expressions:

$$\left. \begin{array}{l} L(r_1 + r_2) = L(r_1) \cup L(r_2) \\ L(r_1.r_2) = L(r_1).L(r_2) \\ L(r_1^*) = (L(r_1))^* \\ L((r_1)) = L(r_1) \end{array} \right\}$$

Regular Expressions and Regular Languages

- **By inductive hypothesis we know:**
 $L(r_1)$ and $L(r_2)$ are regular languages.

Regular Expressions and Regular Languages

- **By inductive hypothesis we know:**
 $L(r_1)$ and $L(r_2)$ are regular languages.
- **we also know:**

Regular languages are closed under

$$\left. \begin{array}{ll} \text{Union} & L(r_1) \cup L(r_2) \\ \text{concatenation} & L(r_1).L(r_2) \\ \text{Star} & (L(r_1))^* \end{array} \right\}$$

Regular Expressions and Regular Languages

- **By inductive hypothesis we know:**

$L(r_1)$ and $L(r_2)$ are regular languages.

- **we also know:**

Regular languages are closed under

$$\left. \begin{array}{ll} \text{Union} & L(r_1) \cup L(r_2) \\ \text{concatenation} & L(r_1).L(r_2) \\ \text{Star} & (L(r_1))^* \end{array} \right\}$$

- **Therefore:**

$$\left. \begin{array}{l} L(r_1 + r_2) = L(r_1) \cup L(r_2) \\ L(r_1.r_2) = L(r_1).L(r_2) \\ L(r_1^*) = (L(r_1))^* \end{array} \right\} = \{\text{are regular languages.}\}$$

Regular Expressions and Regular Languages

- **By inductive hypothesis we know:**

$L(r_1)$ and $L(r_2)$ are regular languages.

- **we also know:**

Regular languages are closed under

$$\left. \begin{array}{ll} \text{Union} & L(r_1) \cup L(r_2) \\ \text{concatenation} & L(r_1).L(r_2) \\ \text{Star} & (L(r_1))^* \end{array} \right\}$$

- **Therefore:**

$$\left. \begin{array}{l} L(r_1 + r_2) = L(r_1) \cup L(r_2) \\ L(r_1.r_2) = L(r_1).L(r_2) \\ L(r_1^*) = (L(r_1))^* \end{array} \right\} = \{\text{are regular languages.}\}$$

- **And trivially:** $L((r_1))$ is a regular language.

Regular Expressions and Regular Languages

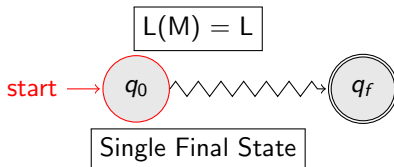
- **Proof of Theorem 2:** For any regular language, $L(r)$, there is a regular expression r with $L(r) = L$.

Regular Expressions and Regular Languages

- **Proof of Theorem 2:** For any regular language, $L(r)$, there is a regular expression r with $L(r) = L$.

Proof by construction of regular expression.

- Since L is regular, take the NFA, M that accepts it.

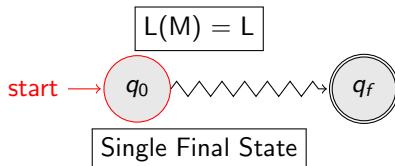


Regular Expressions and Regular Languages

- **Proof of Theorem 2:** For any regular language, $L(r)$, there is a regular expression r with $L(r) = L$.

Proof by construction of regular expression.

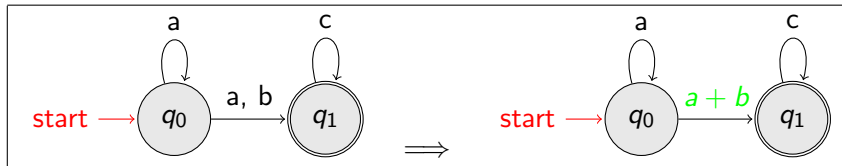
- Since L is regular, take the NFA, M that accepts it.



- From M construct the equivalent **Generalized Transition Graph**: transition labels are regular expressions.

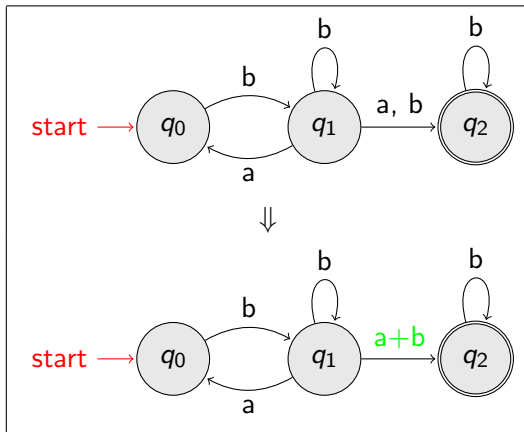
Regular Expressions and Regular Languages

- **Example:** Construct the equivalent Generalized Transition Graph



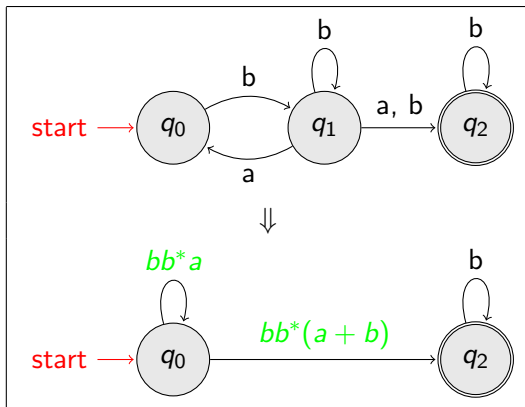
Regular Expressions and Regular Languages

- **Another Example:** Construct the equivalent Generalized Transition Graph



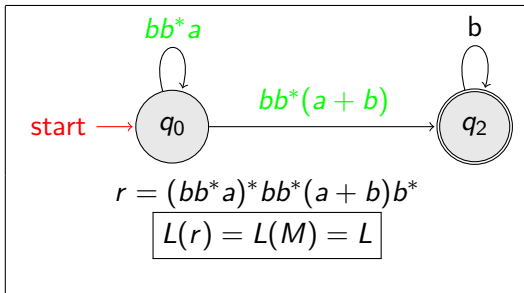
Regular Expressions and Regular Languages

- Reducing the states:



Regular Expressions and Regular Languages

- Resulting Regular Expression:



- Denial I. A. Cohen Introduction to Computer Theory, Second Edition, John Wiley & Sons, 1997.
- J. E. Hopcroft, R. Motwani, & J. D. Ullman Introduction to Automata Theory, Languages, and Computation, Third Edition, Pearson, 2008.